

Università degli Studi “Tor Vergata”

Corso di Laurea in Informatica: Corso di Basi di Dati e di Conoscenza

Esame del 19 gennaio 2026

Domanda 1 (40% della valutazione complessiva)

Una piattaforma online vuole realizzare un **social network per l'organizzazione e la gestione di viaggi di gruppo**, consentendo agli utenti di creare viaggi, partecipare ad esperienze condivise e interagire tra loro.

Il sistema deve rappresentare le seguenti informazioni:

Utenti registrati alla piattaforma

- Ogni **Utente** è identificato univocamente dall'**email**.
Attributi: nome, cognome, dataNascita, telefono, dataRegistrazione.
- Gli utenti possono assumere **ruoli particolari**:

Viaggiatore Premium

- Oltre agli attributi comuni:
 - dataInizioAbbonamento
 - livelloMembership (silver / gold / platinum)
 - creditiViaggio disponibili

Organizzatore

- Oltre agli attributi comuni:
 - biografia
 - lingueParlate
 - punteggioAffidabilità

• Destinazioni

- Ogni **Destinazione** è identificata univocamente da (**nome, paese**).
- Attributi: descrizione, continente, tipo (mare, montagna, città, natura).

• Viaggi di gruppo

- Ogni **Viaggio** è identificato univocamente da (**titolo, organizzatore, dataInizio**).
- Attributi:
 - dataFine
 - costoBase
 - maxPartecipanti
 - stato (pianificato, aperto, completo, annullato)
- Ogni viaggio:
 - è **organizzato da un solo Organizzatore**;
 - fa riferimento ad **una o più Destinazioni**;
 - può includere **esperienze opzionali**.

• Esperienze

- Ogni **Esperienza** è identificata univocamente da (**nome, viaggio**).
- Attributi:
 - descrizione
 - costoAggiuntivo
 - durata
 - livelloDifficoltà (basso / medio / alto)

- **Partecipazioni ai viaggi**

- Una **Partecipazione** è identificata univocamente da (**utente, viaggio**).
- Attributi:
 - dataIscrizione
 - ruoloNelViaggio (partecipante, co-organizzatore)
 - stato (confermata, in attesa, cancellata)
- I **Viaggiatori Premium** possono utilizzare i propri **creditiViaggio** per ottenere:
 - priorità di iscrizione
 - sconti su esperienze opzionali.

- **Recensioni e interazioni sociali**

- Una **Recensione** è identificata univocamente da (**utente, viaggio, dataRecensione**).
- Attributi:
 - voto (1–5)
 - commento
- L'**Organizzatore** del viaggio può rispondere a una recensione:
 - ogni **RispostaRecensione** è identificata da (**recensione, dataRisposta**).
- Tra utenti possono esistere **relazioni sociali**:

- **ConnessioneViaggio**

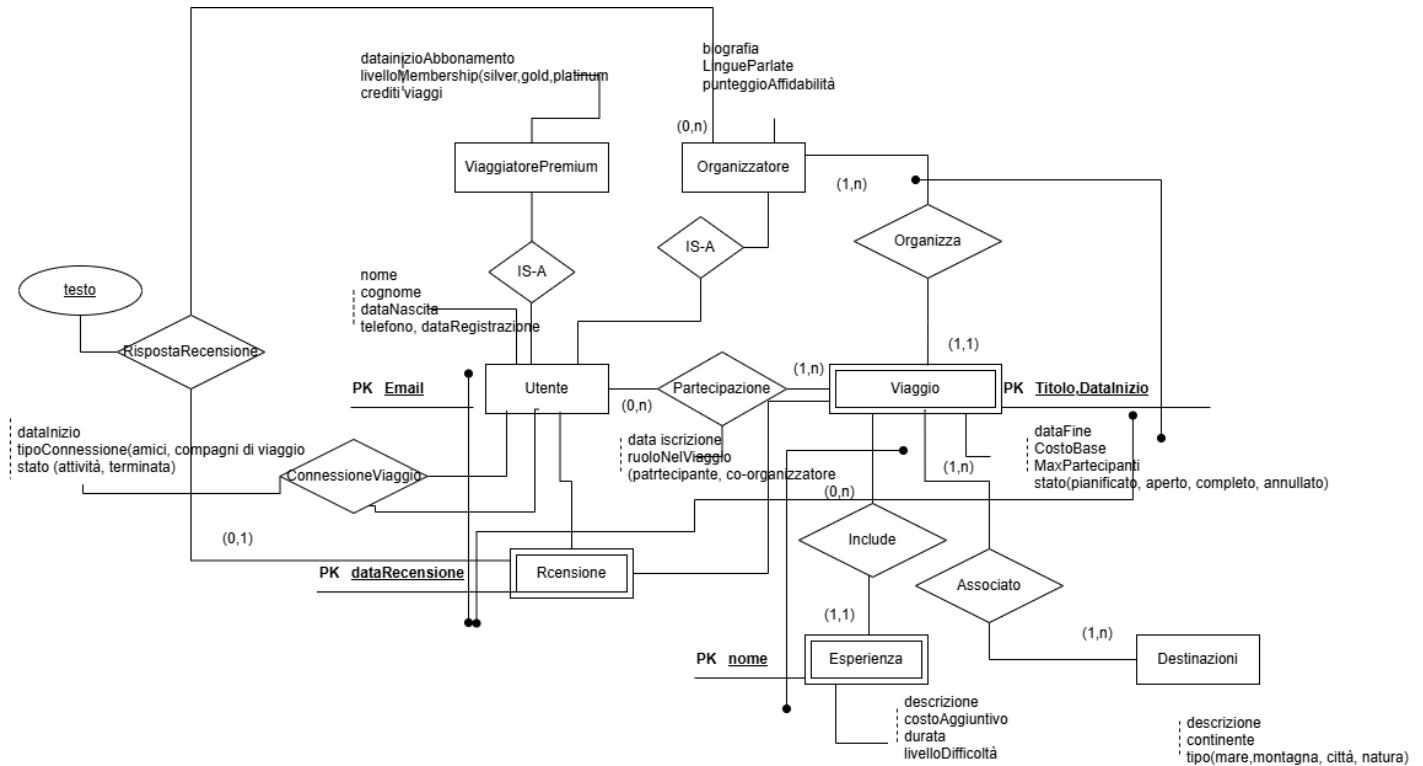
- relazione tra due utenti
- Attributi: dataInizio, tipoConnessione (amici, compagni di viaggio), stato (attiva, terminata)

Richieste

1. **Progettare lo schema concettuale ER** che rappresenti fedelmente la realtà descritta, modellando:
 - le entità e i relativi **identificatori univoci** (chiavi naturali, composte, entità deboli);
 - le **generalizzazioni (IS-A)** tra Utente, Viaggiatore Premium e Organizzatore;
 - le **relazioni con cardinalità complesse**, in particolare:
 - partecipazioni ai viaggi,
 - organizzazione dei viaggi,
 - associazione tra viaggi, destinazioni ed esperienze.
2. **Derivare lo schema relazionale corrispondente**, discutendo le scelte progettuali:
 - rappresentazione delle **gerarchie** (tabelle separate vs tabella unica con discriminatore);
 - introduzione di **identificatori artificiali (ID)** per semplificare:
 - chiavi composte,
 - relazioni molti-a-molti,
 - gestione delle prestazioni;
 - vincoli di unicità sulle **chiavi naturali**

Soluzioni

1. Schema concettuale ER (descrizione)



Entità principali

Utente

- **Chiave naturale:** email
- **Attributi:** nome, cognome, dataNascita, telefono, dataRegistrazione

L'entità **Utente** rappresenta tutti i soggetti registrati alla piattaforma.

Generalizzazione (IS-A) Utente

La generalizzazione è:

- **totale** (tutti gli utenti possono essere almeno base),
- **parziale** rispetto ai ruoli specializzati,
- **sovrapposta** (uno stesso utente può essere sia Viaggiatore Premium sia Organizzatore).

ViaggiatorePremium

- Eredita la chiave email
- **Attributi specifici:**
 - dataInizioAbbonamento
 - livelloMembership (silver, gold, platinum)
 - creditiViaggi

Organizzatore

- Eredita la chiave email
- Attributi specifici:
 - biografia
 - lingueParlate
 - punteggioAffidabilità

Destinazione

- **Chiave naturale composta:** (nome, paese)
- Attributi:
 - descrizione
 - continente
 - tipo (mare, montagna, città, natura)

Viaggio

- **Chiave naturale composta:** (titolo, organizzatore, dataInizio)
- Attributi:
 - dataFine
 - costoBase
 - maxPartecipanti
 - stato (pianificato, aperto, completo, annullato)

Relazioni:

- è **organizzato da** un solo **Organizzatore** (1,N)
- è **associato a** una o più **Destinazioni** (N,M)

Esperienza

- **Entità debole** rispetto a Viaggio
- **Chiave parziale:** nome
- **Chiave completa:** (nome, viaggio)
- Attributi:
 - descrizione
 - costoAggiuntivo
 - durata
 - livelloDifficoltà

Partecipazione

- **Entità associativa**
- **Chiave composta:** (utente, viaggio)
- Attributi:
 - dataIscrizione
 - ruoloNelViaggio (partecipante, co-organizzatore)
 - stato (confermata, in attesa, cancellata)

Recensione

- **Chiave composta:** (utente, viaggio, dataRecensione)
- Attributi:

- voto (1–5)
- commento

RispostaRecensione

- **Entità debole** rispetto a Recensione
- **Chiave composta:** (recensione, dataRisposta)
- Attributi:
 - testoRisposta

ConnessioneViaggio

- **Relazione ricorsiva** su Utente (N,M)
- Attributi:
 - dataInizio
 - tipoConnessione (amici, compagni di viaggio)
 - stato (attiva, terminata)

2. Derivare lo schema relazionale corrispondente,

Per semplificare chiavi e relazioni, si introducono **identificatori artificiali (ID)** mantenendo comunque i **vincoli di unicità sulle chiavi naturali**.

Tabelle principali

- UTENTE(idUtente PK, email UNIQUE, nome, cognome, dataNascita, telefono, dataRegistrazione)
- VIAGGIATORE_PREMIUM(idUtente PK FK → UTENTE, dataInizioAbbonamento, livelloMembership, creditiViaggio)
- ORGANIZZATORE(idUtente PK FK → UTENTE, biografia, lingueParlate, punteggioAffidabilità)
- DESTINAZIONE(idDestinazione PK, nome, paese, descrizione, continente, tipo, UNIQUE(nome, paese))
- VIAGGIO(idViaggio PK, titolo, dataInizio, dataFine, costoBase, maxPartecipanti, stato, idOrganizzatore FK, UNIQUE(titolo, idOrganizzatore, dataInizio))
- VIAGGIO_DESTINAZIONE(idViaggio FK → VIAGGIO, idDestinazione FK → DESTINAZIONE, PK(idViaggio, idDestinazione))
- ESPERIENZA(idEsperienza PK, nome, descrizione, costoAggiuntivo, durata, livelloDifficoltà, idViaggio FK → VIAGGIO, UNIQUE(nome, idViaggio))
- PARTECIPAZIONE(idUtente FK → UTENTE, idViaggio FK → VIAGGIO, dataIscrizione, ruoloNelViaggio, stato, PK(idUtente, idViaggio))
- RECENSIONE(idRecensione PK, voto, commento, dataRecensione, idUtente FK → UTENTE, idViaggio FK → VIAGGIO, UNIQUE(idUtente, idViaggio, dataRecensione))
- RISPOSTA_RECENSIONE(idRisposta PK, testoRisposta, dataRisposta, idRecensione FK → RECENSIONE)
- CONNESSIONE_VIAGGIO(idUtente1 FK → UTENTE, idUtente2 FK → UTENTE, dataInizio, tipoConnessione, stato, PK(idUtente1, idUtente2))

3. Discussione delle scelte progettuali

Gerarchie (IS-A)

Si è scelto di rappresentare la gerarchia Utente / ViaggiatorePremium / Organizzatore tramite:

- tabelle separate per le sottoclassi, collegate tramite PK = FK,

- soluzione più chiara semanticamente e facilmente estendibile.

Identificatori artificiali

Gli ID artificiali:

- semplificano le relazioni N–M,
- evitano chiavi esterne composte complesse,
- migliorano prestazioni e manutenibilità.

Le chiavi naturali sono comunque preservate tramite vincoli UNIQUE.

Entità deboli

- Esperienza e RispostaRecensione dipendono logicamente da altre entità,
- nel relazionale diventano tabelle autonome con FK e vincoli di unicità.

Domanda 2 – (30% della valutazione complessiva)

In base allo **schema relazionale derivato dallo schema ER della domanda precedente**, scrivere le **query in SQL** che rispondono ai seguenti quesiti:

- Elencare **tutti i viaggi** ai quali partecipano **utenti con ruolo di Viaggiatore Premium**, per i quali la **partecipazione risulta confermata** e lo **stato del viaggio è “aperto”**.
- Per **ogni viaggio**, mostrare:
 - il **titolo del viaggio**,
 - il **numero di destinazioni associate**,
 - il **numero totale di partecipanti iscritti** (indipendentemente dallo stato della partecipazione).
- Individuare il nome e il paese della destinazione che compare nel maggior numero di viaggi complessivi. Utilizzare subquery (senza usare LIMIT o costrutti equivalenti).

Algebra relazionale

Scrivere inoltre, in **algebra relazionale**, una query che selezioni **tutte le partecipazioni** relative a **viaggi**:

- che includono almeno una **esperienza con livello di difficoltà “alto”**,
- per i quali l’utente partecipante ha rilasciato una **recensione con voto ≥ 4** .

La query deve restituire almeno:

- l’identificativo dell’utente,
- l’identificativo del viaggio.

Soluzione

- Elencare tutti i viaggi ai quali partecipano utenti con ruolo di Viaggiatore Premium, per i quali la partecipazione risulta confermata e lo stato del viaggio è “aperto”.**

Logica

- un Viaggiatore Premium è presente in VIAGGIATORE_PREMIUM
- la partecipazione è in PARTECIPAZIONE
- lo stato del viaggio è in VIAGGIO

Query SQL

```
SELECT DISTINCT v.idViaggio, v.titolo, v.dataInizio, v.dataFine
```

```

FROM VIAGGIO v
JOIN PARTECIPAZIONE p
    ON v.idViaggio = p.idViaggio
JOIN VIAGGIATORE_PREMIUM vp
    ON p.idUtente = vp.idUtente
WHERE p.stato = 'confermata'
    AND v.stato = 'aperto';

```

2. Per ogni viaggio, mostrare:

il titolo del viaggio,

- **il numero di destinazioni associate,**
- **il numero totale di partecipanti iscritti (indipendentemente dallo stato della partecipazione).**

Logica

- destinazioni: VIAGGIO_DESTINAZIONE
- partecipanti: PARTECIPAZIONE
- uso di LEFT JOIN per includere anche viaggi senza destinazioni o partecipanti

Query SQL

```

SELECT v.idViaggio, v.titolo,
       COUNT(DISTINCT vd.idDestinazione) AS numeroDestinazioni,
       COUNT(DISTINCT p.idUtente) AS numeroPartecipanti
FROM VIAGGIO v
LEFT JOIN VIAGGIO_DESTINAZIONE vd
    ON v.idViaggio = vd.idViaggio
LEFT JOIN PARTECIPAZIONE p
    ON v.idViaggio = p.idViaggio
GROUP BY v.idViaggio, v.titolo;

```

3. Individuare il nome e il paese della destinazione che compare nel maggior numero di viaggi complessivi.

Utilizzare subquery (senza usare LIMIT o costrutti equivalenti).

Logica

- contare in quanti viaggi compare ogni destinazione
- selezionare quella con il valore massimo

Query SQL

```

SELECT d.nome, d.paese
FROM DESTINAZIONE d
JOIN VIAGGIO_DESTINAZIONE vd
    ON d.idDestinazione = vd.idDestinazione
GROUP BY d.idDestinazione, d.nome, d.paese
HAVING COUNT(vd.idViaggio) >= ALL (
    SELECT COUNT(vd2.idViaggio)
    FROM VIAGGIO_DESTINAZIONE vd2
    GROUP BY vd2.idDestinazione
);

```

Scrivere inoltre, in **algebra relazionale**, una query che selezioni **tutte le partecipazioni** relative a **viaggi**:

- che includono almeno una **esperienza con livello di difficoltà “alto”**,
- per i quali l’utente partecipante ha rilasciato **una recensione con voto ≥ 4** .

La query deve restituire almeno:

- l’identificativo dell’utente,
- l’identificativo del viaggio.

Costruzioni intermedie

1. Esperienze con difficoltà alta

$E_alto \leftarrow \sigma \text{ livelloDifficoltà} = 'alto' \text{ (ESPERIENZA)}$

2. Viaggi che includono almeno una esperienza “alta”

$V_alto \leftarrow \pi \text{ idViaggio (E_alto)}$

3. Recensioni con voto ≥ 4

$R_buone \leftarrow \sigma \text{ voto} \geq 4 \text{ (RECENSIONE)}$

4. Partecipazioni con recensione positiva

$P_rec \leftarrow \text{PARTECIPAZIONE} \bowtie_{\text{PARTECIPAZIONE.idUtente} = \text{RECENSIONE.idUtente} \wedge \text{PARTECIPAZIONE.idViaggio} = \text{RECENSIONE.idViaggio}} R_buone$

5. Selezione delle partecipazioni relative a viaggi con esperienza “alta”

$\text{Risultato} \leftarrow P_rec \bowtie_{P_rec.idViaggio = V_alto.idViaggio} V_alto$

6. Proiezione finale

$\pi \text{ idUtente, idViaggio (Risultato)}$

Domanda 3 (30% della valutazione)

Un centro commerciale raccoglie i dati relativi alle vendite effettuate nei vari negozi e memorizza le informazioni nella seguente relazione non normalizzata:

VENDITA_CC(ID_Scontrino, DataVendita, ID_Cliente, NomeCliente, ID_Negozi, NomeNegozio, CategoriaNegozio, Piano, ID_Prodotto, NomeProdotto, PrezzoUnitario, Quantità, ID_Addetto, NomeAddetto)

Significato degli attributi:

- **ID_Scontrino**: identificativo univoco della vendita.
- **DataVendita**: data in cui è avvenuta la vendita.
- **ID_Cliente**: identificativo univoco del cliente.
- **NomeCliente**: nome e cognome del cliente.
- **ID_Negozi**: identificativo univoco del negozio nel centro commerciale.
- **NomeNegozio**: denominazione del negozio.
- **CategoriaNegozio**: tipologia del negozio (es. abbigliamento, elettronica, ristorazione).
- **Piano**: piano del centro commerciale in cui si trova il negozio.
- **ID_Prodotto**: identificativo univoco del prodotto.
- **NomeProdotto**: nome del prodotto venduto.
- **PrezzoUnitario**: prezzo unitario del prodotto.

- **Quantità:** numero di unità vendute del prodotto.
- **ID_Addetto:** identificativo dell'addetto alla vendita.
- **NomeAddetto:** nome e cognome dell'addetto.

Vincoli noti

- Ogni **cliente** può effettuare più acquisti nel tempo.
- Ogni **negozio** appartiene a una sola categoria ed è situato in un solo piano del centro commerciale.
- Ogni **addetto** lavora in un solo negozio.
- Ogni **prodotto** è venduto da un solo negozio, ma può comparire in più vendite.
- Uno **scontrino** può includere più prodotti.

Domande

1. **Elencare tutte le dipendenze funzionali** presenti nello schema dato.
2. **Individuare la chiave primaria** della relazione non normalizzata, motivando la scelta.
3. **Trasformare la relazione:**
 - in **Prima Forma Normale (1NF)**,
 - successivamente in **Seconda Forma Normale (2NF)**,
 - e infine in **Terza Forma Normale (3NF)**,
 evidenziando chiaramente **in quale passaggio emerge una dipendenza transitiva** che rende necessaria la decomposizione in 3NF ma non in 2NF.
4. **Verificare se lo schema ottenuto in 3NF soddisfa anche la BCNF.**
 In caso contrario, indicare:
 - quali dipendenze violano la BCNF,
 - quali ulteriori decomposizioni sono necessarie.
5. **Riportare lo schema relazionale finale in BCNF**, specificando per ogni relazione:
 - chiave primaria,
 - eventuali chiavi esterne.

Soluzione

Relazione iniziale (non normalizzata)

VENDITA_CC(ID_Scontrino, DataVendita, ID_Cliente, NomeCliente, ID_Negozio, NomeNegozio, CategoriaNegozio, Piano, ID_Prodotto, NomeProdotto, PrezzoUnitario, Quantità, ID_Addetto, NomeAddetto)

Dipendenze funzionali

1. ID_Scontrino → DataVendita, ID_Cliente, ID_Addetto
2. ID_Cliente → NomeCliente
3. ID_Addetto → NomeAddetto, ID_Negozio
4. ID_Negozio → NomeNegozio, CategoriaNegozio, Piano
5. ID_Prodotto → NomeProdotto, PrezzoUnitario
6. (ID_Scontrino, ID_Prodotto) → Quantità

Chiave primaria

(ID_Scontrino, ID_Prodotto)

Trasformazione in 1NF

La relazione è già in 1NF (attributi atomici).

Trasformazione 2NF – Eliminazione dipendenze parziali

Dipendenza che viola la 2NF

ID_Scontrino → DataVendita, ID_Cliente, ID_Addetto

VENDITA(ID_Scontrino PK, DataVendita, ID_Cliente, ID_Addetto)

Dipendenza che viola la 2NF

ID_Prodotto → NomeProdotto, PrezzoUnitario

PRODOTTO(ID_Prodotto PK, NomeProdotto, PrezzoUnitario)

Dipendenza che viola la 2NF

(ID_Scontrino, ID_Prodotto) → Quantità

DETTAGLIO_VENDITA(ID_Scontrino PK, ID_Prodotto PK, Quantità)

Tabella in 2NF

VENDITA_CC(ID_Scontrino, ID_Cliente, NomeCliente, ID_Negoziio, NomeNegoziio, CategoriaNegoziio, Piano, ID_Prodotto, ID_Addetto, NomeAddetto)

Trasformazione 3NF – Eliminazione dipendenze transitive

Dipendenze transitive per la tabella

VENDITA_CC(ID_Scontrino, ID_Cliente, NomeCliente, ID_Negoziio, NomeNegoziio, CategoriaNegoziio, Piano, ID_Prodotto, ID_Addetto, NomeAddetto)

- ID_Scontrino → ID_Cliente → NomeCliente
- ID_Addetto → ID_Negoziio → NomeNegoziio, CategoriaNegoziio, Piano

Nuove tabelle

CLIENTE(ID_Cliente PK, NomeCliente)

NEGOZIO(ID_Negoziio PK, NomeNegoziio, CategoriaNegoziio, Piano)

ADDETTO(ID_Addetto PK, NomeAddetto, ID_Negoziio FK)

Verifica BCNF

✓ In tutte le relazioni, il determinante di ogni dipendenza funzionale è una **chiave**

Schema Finale

```
CLIENTE(ID Cliente PK, NomeCliente )
NEGOZIO( ID Negozio PK, NomeNegozio, CategoriaNegozio, Piano )
ADDETTO(ID Addetto PK, NomeAddetto, ID_Negozio)
PRODOTTO( ID Prodotto PK, NomeProdotto, PrezzoUnitario)
VENDITA(ID Scontrino PK, DataVendita, ID_Cliente, ID_Addetto)
DETTAGLIO_VENDITA(ID Scontrino PK, ID Prodotto PK, Quantità)
```